وثيقة المراجعة والتلخيص الأكاديمي

Software Design - Chapter 5 - Solved

PART A - Multiple Choice Questions (5.1 to 5.12)

5.1 The most desirable form of coupling is: (a) Control Coupling (b) Data Coupling (c) Common Coupling (d) Content Coupling

Answer: (b) Data Coupling

Data coupling is the lowest (best) form of coupling, where two modules communicate only by passing primitive data items and are otherwise independent.

الترجمة: أفضل شكل اقتران هو الاقتران بالبيانات، لأنه أقل شكل - وحدتان تتواصلان فقط بتمرير بيانات أولية وما عدا ذلك مستقلتان.

5.2 The worst type of coupling is: (a) Content coupling (b) Common coupling (c) External coupling (d) Data coupling

Answer: (a) Content coupling

Content coupling is the highest (worst) form, where one module changes data of another module or branches into the middle of it.

الترجمة: أسوأ نوع اقتران هو الاقتران بالمحتوى، لأن وحدة تغيّر بيانات وحدة ثانية أو تدخل في منتصفها.

5.3 The most desirable form of cohesion is: (a) Logical cohesion (b) Procedural cohesion (c) Functional cohesion (d) Temporal cohesion

Answer: (c) Functional cohesion

Functional cohesion is the best possible cohesion, where all functions of the module cooperate to complete a single task.

الترجمة: أفضل شكل ترابط هو الترابط الوظيفي، لأن كل دوال الوحدة تتعاون لإنجاز مهمة واحدة.

5.4 The worst type of cohesion is: (a) Temporal cohesion (b) Coincidental cohesion (c) Logical cohesion (d) Sequential cohesion

Answer: (b) Coincidental cohesion

Coincidental cohesion is the worst, where the module contains a random collection of functions with little or no relationship.

الترجمة: أسوأ نوع ترابط هو الترابط العشوائي، لأن الوحدة تحتوي تجميعة عشوائية من الدوال بلا علاقة بينها.

5.5 Which one is not a strategy for design? (a) Bottom up design (b) Top down design (c) Embedded design (d) Hybrid design

Answer: (c) Embedded design

The three design strategies are Bottom-up, Top-down, and Hybrid. Embedded design is not one of them.

الترجمة: الاستراتيجية اللي مو من استراتيجيات التصميم هي التصميم المدمج، لأن الاستراتيجيات الثلاث هي من الأسفل للأعلى، من الأعلى للأسفل، والمختلطة.

5.6 Temporal cohesion means: (a) Cohesion between temporary variables (b) Cohesion between local variable (c) Cohesion with respect to time (d) Coincidental cohesion

Answer: (c) Cohesion with respect to time

Temporal cohesion occurs when a module contains tasks that must all be executed in the same time span, such as initialisation, setup, and shutdown.

الترجمة: الترابط الزمني يعني الترابط بالنسبة للوقت، لأن المهام لازم تُنفَّذ كلها في نفس الفترة الزمنية مثل التهيئة والإعداد والإيقاف.

5.7 Functional cohesion means: (a) Operations are part of single functional task and are placed in same procedures (b) Operations are part of single functional task and are placed in multiple procedures (c) Operations are part of multiple tasks (d) None of the above

Answer: (a) Operations are part of a single functional task and are placed in the same procedure.

Functional cohesion means all operations cooperate for one single task and are contained together, and the module can be described by one simple sentence.

الترجمة: الترابط الوظيفي يعني أن العمليات جزء من مهمة وظيفية واحدة وتُوضع في نفس الإجراء، وكل العمليات تتعاون لمهمة واحدة توصف بجملة واحدة.

5.8 When two modules refer to the same global data area, they are related as: (a) External coupled (b) Data coupled (c) Content coupled (d) Common coupled

Answer: (d) Common coupled

Common coupling exists when two modules share global (shared) data areas, and any change in the common data must be traced back to all modules accessing it.

الترجمة: عندما تشير وحدتان لنفس منطقة البيانات الشاملة فهما مقترنتان اقترانًا مشتركًا، وأي تغيير بالبيانات الشاملة لازم يُتتبَّع لكل الوحدات اللي تدخلها.

5.9 The module in which instructions are related through flow of control is: (a) Temporal cohesion (b) Logical cohesion (c) Procedural cohesion (d) Functional cohesion

Answer: (c) Procedural cohesion

Procedural cohesion occurs when the instructions of a module, although accomplishing different tasks, have been combined because there is a specific order in which the tasks must be completed.

الترجمة: الوحدة اللي تعليماتها مرتبطة عبر تدفق السيطرة هي الترابط الإجرائي، لأن التعليمات رغم أنها تؤدي مهامًا مختلفة جُمعت بسبب وجود ترتيب محدد لإنجازها.

5.10 The relationship of data elements in a module is called: (a) Coupling (b) Cohesion (c) Modularity (d) None of the above

Answer: (b) Cohesion

Cohesion measures the degree to which the elements within a module are functionally related, while coupling measures the relationship between modules.

الترجمة: علاقة عناصر البيانات داخل الوحدة تُسمّى الترابط، لأنه يقيس مدى ارتباط العناصر داخل الوحدة نفسها، بعكس الاقتران اللي يقيس العلاقة بين الوحدات.

5.11 A system that does not interact with external environment is called: (a) Closed system (b) Logical system (c) Open system (d) Hierarchal system

Answer: (a) Closed system

A closed system does not interact with the external environment, whereas an open system does.

الترجمة: النظام اللي ما يتفاعل مع البيئة الخارجية يُسمّى النظام المغلق، بينما النظام المفتوح يتفاعل مع البيئة.

5.12 The extent to which different modules are dependent upon each other is called: (a) Coupling (b) Cohesion (c) Modularity (d) Stability

Answer: (a) Coupling

Coupling is the measure of the degree of interdependence between modules.

الترجمة: مدى اعتماد الوحدات المختلفة على بعضها يُسمّى الاقتران، وهو مقياس درجة الاعتماد المتبادل بين الوحدات.

PART B - Essay Questions (5.1 to 5.24)

5.1 What is design? Describe the difference between conceptual design and technical design.

Answer: Design is the process of translating requirements into a description of the software structure, components, interfaces and data needed for implementation. Its output is the Software Design Document (SDD). Conceptual design answers the what: where data comes from, what happens to the data, how the system looks to users, what choices are offered, the timing of events and how reports and screens look. Technical design describes the how: hardware configuration, software needs, communication interfaces, input/output of the system, software architecture, network architecture and anything that translates the requirements into a solution to the customer's problem.

الترجمة: التصميم هو عملية ترجمة المتطلبات إلى وصف لبنية البرمجيات ومكوناتها وواجهاتها وبياناتها اللازمة للتنفيذ، ومخرجه مستند التصميم SDD. التصميم المفهومي يجيب على سؤال ماذا: منين تجي البيانات وشنو يصير بيها وشلون يبدو النظام للمستخدم وشنو الخيارات المطروحة وتوقيت الأحداث. أما التصميم التقني فيصف كيف: تهيئة العتاد واحتياجات البرمجيات وواجهات الاتصال والمدخلات والمخرجات ومعمارية البرمجيات والشبكة، وأي شي يترجم المتطلبات إلى حل لمشكلة العميل.

5.2 Discuss the objectives of software design. How do we transform an informal design to a detailed design?

Answer: The design needs to be correct and complete, understandable, at the right level, and maintainable. The transformation from an informal to a detailed design happens through stepwise refinement: we start with an informal design outline, then an informal design, then a more formal design, and finally a finished design. At each step the design is refined to a more concrete level until no more refinement is needed and the design can be implemented directly. Abstraction and refinement are complementary concepts.

الترجمة: لازم يكون التصميم صحيحًا وكاملًا ومفهومًا وبالمستوى المناسب وقابلًا للصيانة. والتحول من التصميم غير الرسمي إلى التفصيلي يصير عبر التنقيح التدريجي: نبدأ بمسودة غير رسمية ثم تصميم غير رسمي ثم تصميم أكثر رسمية وأخيرًا تصميم

منتهي، وفي كل خطوة يُنقّح التصميم لمستوى أكثر تحديدًا حتى نوصل لمستوى يمكن تنفيذه مباشرة. والتجريد والتنقيح مفهومان متكاملان.

5.3 Do we design software when we write a program? What makes software design different from coding?

Answer: Yes, some design happens when writing a program, but design is not coding and coding is not design. Design is the place where quality is built into the software; it is more creative than analysis and it answers the question how, working at a higher level of abstraction than source code. The only design decisions made at the coding level address the small implementation details that let the design be coded. Design should be traceable to the analysis model and it serves as the blueprint for construction.

الترجمة: نعم، جزء من التصميم يصير عند كتابة البرنامج، لكن التصميم مو ترميز والترميز مو تصميم. التصميم هو المكان اللي تُبنى فيه الجودة، وهو أكثر إبداعًا من التحليل ويجيب على سؤال كيف، بمستوى تجريد أعلى من الكود المصدري. القرارات التصميمية الوحيدة اللي تتخذ بمستوى الترميز هي تفاصيل التنفيذ الصغيرة اللي تسمح بترميز التصميم. والتصميم يجب أن يكون قابلًا للتتبع إلى نموذج التحليل ويُعدّ مخططًا للبناء.

5.4 What is modularity? List the important properties of a modular system.

Answer: A module ranges from a subroutine to a package to a class to a work assignment for one programmer, and all these definitions are correct. A modular system consists of well-defined, manageable units with well-defined interfaces among the units. Its properties are: each function has a single well-defined purpose; each function manipulates no more than one major data structure; functions share global data selectively; and functions that manipulate abstract data types are encapsulated with the data structure. Modularity is the single attribute that allows a program to be intellectually manageable, and it eases implementation, debugging, testing, documenting and maintenance.

الترجمة: الوحدة تتراوح من روتين فرعي إلى حزمة إلى صنف إلى مهمة لمبرمج واحد، وكل هذه التعريفات صحيحة. النظام الموديولي يتكون من وحدات مُدارة ومُعرّفة جيدًا بواجهات محددة بينها. وخصائصه: كل دالة لها غرض واحد مُعرّف، وكل دالة تتعامل مع بنية بيانات رئيسية واحدة على الأكثر، والدوال تتشارك البيانات الشاملة بشكل انتقائي، والدوال اللي تتعامل مع أنواع البيانات المجردة تكون مغلّفة مع بنية البيانات. والـ modularity هو الصفة الوحيدة اللي تجعل البرنامج قابلًا للإدارة الذهنية، وتسهّل التنفيذ والتنقيح والاختبار والتوثيق والصيانة.

5.5 Define module coupling and explain different types of coupling.

Answer: Coupling is the measure of the degree of interdependence between modules, and it depends on the interface complexity. From best to worst the types are: Data coupling, where modules communicate by passing only an elementary data item; Stamp coupling, where a composite data item such as a record or structure is passed; Control coupling, where control information such as a flag is passed to direct the order of execution; External coupling, where a module is tied to an external environment such as I/O; Common coupling, where modules share a global data area; and Content coupling, where one module changes data of another or branches into the middle of it. We always strive for the lowest possible coupling.

الترجمة: الاقتران هو مقياس درجة الاعتماد المتبادل بين الوحدات، ويعتمد على تعقيد الواجهة. ومن الأفضل للأسوأ: الاقتران بالبيانات حيث تتواصل الوحدات بتمرير عنصر بيانات أولي فقط، والاقتران بالطابع حيث تُمرَّر بنية بيانات كاملة مثل سجل، والاقتران بالتحكم حيث تُمرَّر معلومة تحكم مثل flag لتوجيه ترتيب التنفيذ، والاقتران الخارجي حيث ترتبط وحدة ببيئة خارجية مثل المدخلات والمخرجات، والاقتران المشترك حيث تتشارك الوحدات منطقة بيانات شاملة، والاقتران بالمحتوى حيث تغيّر وحدة بيانات وحدة أخرى أو تدخل في منتصفها. ونحن دائمًا نسعى لأقل اقتران ممكن.

5.6 Define module cohesion and explain different types of cohesion.

Answer: Cohesion is a measure of the functional strength of a module, that is, the degree to which the elements of a module are functionally related. From best to worst the types are: Functional cohesion, where all functions cooperate to complete a single task; Sequential cohesion, where the output of one function is the input to the next; Communicational cohesion, where all functions operate on the same data structure; Procedural cohesion, where functions are executed one after another in a specific order; Temporal cohesion, where functions are related by being executed in the same time span; Logical cohesion, where functions belong to the same logical class such as all input or all error handling; and Coincidental cohesion, where the functions have little or no relationship. We always strive for high cohesion.

الترجمة: الترابط هو مقياس القوة الوظيفية للوحدة، أي مدى ارتباط عناصر الوحدة وظيفيًا. ومن الأفضل للأسوأ: الترابط الوظيفي حيث كل الدوال تتعاون لإنجاز مهمة واحدة، والترابط التسلسلي حيث مخرج دالة هو مدخل الدالة التالية، والترابط التواصلي حيث كل الدوال تعمل على نفس بنية البيانات، والترابط الإجرائي حيث تُنفَّذ الدوال واحدة بعد الأخرى بترتيب محدد، والترابط الزمني حيث ترتبط الدوال بأنها تُنفَّذ في نفس الفترة الزمنية، والترابط المنطقي حيث تنتمي الدوال لنفس الصنف المنطقي مثل كل الإدخال أو كل معالجة الأخطاء، والترابط العشوائي حيث لا علاقة بين الدوال. ونحن دائمًا نسعى لترابط عالٍ.

5.7 Discuss the objectives of modular software design. What are the effects of module coupling and cohesion?

Answer: The objective of modular design is functional independence, which is achieved when a module is highly cohesive and has low coupling with other modules. Functional independence gives three main advantages: error isolation, because an error in a functionally independent module is unlikely to propagate and is easy to locate; scope of reuse, because a well-defined module with simple interfaces can easily be taken out and reused in another program; and understandability, because independent modules can be understood in isolation, reducing design complexity. High coupling causes a ripple effect where an error in one location propagates through the system, and low cohesion makes the module hard to understand, test and reuse. Functional independence is a key to good design, and design is the key to software quality.

الترجمة: هدف التصميم الموديولي هو الاستقلال الوظيفي، ويتحقق عندما تكون الوحدة عالية الترابط وقليلة الاقتران مع الوحدات الأخرى. والاستقلال الوظيفي يعطي ثلاث فوائد رئيسية: عزل الأخطاء لأن خطأ وحدة مستقلة لا ينتشر ويسهل تحديده، ومجال إعادة الاستخدام لأن الوحدة المُعرّفة جيدًا وبواجهات بسيطة يسهل سحبها وإعادة استخدامها في برنامج آخر، وقابلية الفهم لأن الوحدات المستقلة تُفهم منعزلًا فيقل تعقيد التصميم. أما الاقتران العالي فيسبب تأثير التموّج حيث ينتشر خطأ موقع واحد في النظام، والترابط المنخفض يجعل الوحدة صعبة الفهم والاختبار وإعادة الاستخدام. والاستقلال الوظيفي مفتاح التصميم الجيد، والتصميم مفتاح جودة البرمجيات.

5.8 If a module has logical cohesion, what kind of coupling is this module likely to have with others?

Answer: A module with logical cohesion groups functions that are similar in a logical sense, such as all input or all output functions. To distinguish which function to perform, such a module needs a control signal passed to it, so it is likely to have control coupling with other modules. It may even reach common coupling if it depends on shared global data that determines its mode of operation, because the flag that selects the function has to be set by one module and reacted upon by the dependent module.

الترجمة: الوحدة ذات الترابط المنطقي تجمع دوالًا متشابهة منطقيًا مثل كل دوال الإدخال أو كل دوال الإخراج. ولتمييز أي دالة تُنفَّذ تحتاج إشارة تحكم تُمرَّر إليها، فغالبًا يكون لديها اقتران بالتحكم مع الوحدات الأخرى، وقد تصل إلى الاقتران المشترك إذا اعتمدت على بيانات شاملة تحدد وضع تشغيلها، لأن الـ flag اللي يختار الدالة لازم تضبطه وحدة وتتفاعل معه الوحدة المعتمدة.

5.9 What problems are likely to arise if two modules have high coupling?

Answer: High coupling causes several problems. First, a ripple effect, because an error in one location propagates through the system. Second, difficulty in debugging, because the actual cause of a failure may be in a different module from where the failure appears, as when one module updates a shared item incorrectly and another module fails later when it reads it. Third, difficulty in reuse, because a highly coupled module cannot easily be taken out and inserted into another program. Fourth, fragility, because small changes to requirements may cause systemwide side effects. Fifth, difficulty in understanding, because complex interfaces raise the intellectual load.

الترجمة: الاقتران العالي يسبب عدة مشاكل: أولًا تأثير التموّج لأن خطأ موقع واحد ينتشر في النظام، وثانيًا صعوبة التنقيح لأن السبب الحقيقي للفشل قد يكون في وحدة مختلفة عن مكان ظهوره كما لو حدّثت وحدة عنصرًا مشتركًا بشكل خاطئ ثم فشلت وحدة أخرى عندما قرأته، وثالثًا صعوبة إعادة الاستخدام لأن الوحدة المترابطة يصعب سحبها وإدخالها في برنامج آخر، ورابعًا الهشاشة لأن تغييرات صغيرة بالمتطلبات قد تسبب آثارًا جانبية على مستوى النظام، وخامسًا صعوبة الفهم لأن الواجهات المعقدة ترفع الحمل الذهني.

5.10 What problems are likely to arise if a module has low cohesion?

Answer: Low cohesion causes several problems. First, coupling increases, because if a function is moved out of the module, the functions then interact across two different modules. Second, error propagation, because a module that groups independent functions will have any modification to one of them affect the others. Third, loss of understandability and reuse, because the module cannot be described by a simple sentence and so is hard to maintain or move. Fourth, difficulty in testing, because a module performing unrelated functions makes test cases branch out.

الترجمة: الترابط المنخفض يسبب عدة مشاكل: أولًا ازدياد الاقتران لأن نقل أي دالة من الوحدة يجعل الدوال تتفاعل عبر وحدتين مختلفتين، وثانيًا انتشار الأخطاء لأن الوحدة اللي تجمع دوالًا مستقلة سيؤثر أي تعديل على إحداها في الباقي، وثالثًا فقدان قابلية الفهم وإعادة الاستخدام لأن الوحدة ما توصف بجملة واحدة فتصعب صيانتها أو نقلها، ورابعًا صعوبة الاختبار لأن الوحدة اللي تؤدي وظائف غير مترابطة تُشعّب حالات الاختبار.

5.11 Describe the various strategies of design. Which design strategy is most popular and practical?

Answer: There are three strategies. Top-down design starts by identifying the major modules and decomposing them into lower-level modules, refining step by step until the statement level is reached. Bottom-up design starts with small modules that are collected together in the form of a library. Hybrid design combines both. Hybrid is the most popular and practical, because a purely top-down approach is not enough: a bottom-up approach is essential to permit common sub-modules, because near the bottom of the hierarchy the intuition is simpler and the need for bottom-up testing is greater as there are more modules at low levels, and because of the use of pre-written library modules and reuse.

الترجمة: توجد ثلاث استراتيجيات: من الأعلى للأسفل تبدأ بتحديد الوحدات الرئيسية وتفكيكها لوحدات أدنى مستوى وتنقيحها خطوة بخطوة حتى مستوى الجملة، ومن الأسفل للأعلى تبدأ بوحدات صغيرة تُجمع على شكل مكتبة، والمختلطة تجمع الاثنين. والمختلطة هي الأكثر شيوعًا وعملية، لأن النهج من الأعلى للأسفل وحده لا يكفي: نحتاج نهجًا من الأسفل للأعلى للسماح بالوحدات الفرعية المشتركة، ولأن الحدس أبسط قرب أسفل الهرم والحاجة للاختبار من الأسفل أكبر لوجود وحدات أكثر بالمستويات الدنيا، ولأجل استخدام وحدات المكتبة الجاهزة وإعادة الاستخدام.

5.12 If some existing modules are to be re-used in building a new system, which design strategy is used and why?

Answer: Bottom-up design, or the bottom-up part of a hybrid approach, is used. The reason is that pre-written library modules are combined from the bottom upwards: small existing modules are collected together in the form of a library and then higher-level modules are built on top of them. The use of pre-written library modules

and the reuse of modules is one of the stated reasons why a bottom-up approach is essential even inside a top-down design.

الترجمة: تُستخدم الاستراتيجية من الأسفل للأعلى، أو الجزء السفلي من النهج المختلط. والسبب أن وحدات المكتبة الجاهزة تُجمَّع من الأسفل للأعلى: وحدات صغيرة موجودة تُجمع على شكل مكتبة ثم تُبنى فوقها وحدات أعلى مستوى. واستخدام وحدات المكتبة الجاهزة وإعادة استخدام الوحدات أحد الأسباب المذكورة لضرورة النهج من الأسفل للأعلى حتى داخل التصميم من الأعلى للأسفل.

5.13 What is the difference between a flow chart and a structure chart?

Answer: A flow chart represents the sequence of steps and the flow of control, that is, the logic of an algorithm at a detailed, step-by-step level. A structure chart represents the hierarchical organisation of modules and their interconnections, and it is used during architectural design to document hierarchical structures, parameters and interconnections. In a structure chart there is one and only one module at the top called the root, and the components are read from top to bottom and left to right; it partitions a system into black boxes. Its basic building blocks are rectangular boxes for modules, arrows for control flow, data-flow arrows, library modules, selection symbols and repetition symbols.

الترجمة: مخطط سير العمليات يمثل تسلسل الخطوات وتدفق السيطرة، أي منطق الخوارزمية بمستوى تفصيلي خطوة بخطوة. أما مخطط البنية فيمثل التنظيم الهرمي للوحدات وترابطاتها، ويُستخدم في التصميم المعماري لتوثيق البنى الهرمية والمعاملات والترابطات. وفي مخطط البنية توجد وحدة واحدة فقط في الأعلى تُسمى الجذر، وتُقرأ المكونات من الأعلى للأسفل ومن اليسار لليمين، وهو يقسّم النظام إلى صناديق سوداء. ومكوناته الأساسية: مربعات للوحدات، وأسهم لتدفق التحكم، وأسهم لتدفق البيانات، ووحدات مكتبية، ورموز اختيار، ورموز تكرار.

5.14 Explain why it is important to use different notations to describe software designs.

Answer: Different notations are important because each stakeholder and each aspect of the design needs a suitable representation, and using several notations reduces ambiguity. A structure chart documents the hierarchical structure and interfaces; pseudocode, also called program-design language or structured English, describes detailed logic and is easy to convert to a programming language and easy to modify; data flow diagrams and data dictionaries describe the data and its flow. Using several notations serves different audiences such as the programmer, the tester and the designer, and it makes review more accurate because each representation reveals a different kind of error. Design notations are largely meant to be used during the process of design to represent design or design decisions.

الترجمة: الصيغ المختلفة مهمة لأن كل صاحب مصلحة وكل جانب من التصميم يحتاج تمثيلًا مناسبًا، واستخدام عدة صيغ يقلل الغموض. مخطط البنية يوثّق البنية الهرمية والواجهات، والـ pseudocode المعروف بلغة تصميم البرامج أو الإنجليزية المهيكلة يوصف المنطق التفصيلي ويسهل تحويله للغة برمجة ويسهل تعديله، ومخططات تدفق البيانات وقواميس البيانات تصف البيانات وتدفقها. واستخدام عدة صيغ يخدم جماهير مختلفة كمبرمج ومختبِر ومصمم، ويجعل المراجعة أدق لأن كل تمثيل يكشف نوعًا مختلفًا من الأخطاء. وصيغ التصميم تُستخدم غالبًا أثناء عملية التصميم لتمثيل التصميم أو قراراته.

5.15 List a few well-established function oriented software design techniques.

Answer: The well-established function-oriented design techniques are: Structured design by Constantine and Yourdon (1979); Jackson's structured design by Jackson (1975); the Warnier-Orr methodology (1977, 1981); step-wise refinement by Wirth (1971); and Hatley and Pirbhai's methodology (1987). Function-oriented design is the approach where the design is decomposed into a set of interacting units, each with a clearly defined function, so the system is designed from a functional viewpoint.

الترجمة: تقنيات التصميم الوظيفي المعتمدة هي: التصميم المهيكل من Constantine و Yourdon سنة 1979، والتصميم المهيكل لجاكسون سنة 1975، ومنهجية Warnier-Orr سنتي 1977 و1981، والتنقيح التدريجي من Wirth سنة 1971، ومنهجية Hatley و Pirbhai سنة 1987. والتصميم الوظيفي هو النهج اللي يُفكَّك فيه التصميم إلى مجموعة وحدات متحاورة كل واحدة بدالة معرّفة بوضوح، فيُصمَّم النظام من منظور وظيفي.

5.16 Define the following terms: Objects, Message, Abstraction, Class, Inheritance and Polymorphism.

Answer: Object: an entity able to save a state and offer operations to examine or affect this state. Message: the way objects communicate, consisting of the identity of the target object, the name of the requested operation and any other information needed. Abstraction: the elimination of the irrelevant and the amplification of the essentials. Class: a set of objects that share a common structure and a common behaviour, acting as a blueprint for objects. Inheritance: defining a new class by incrementally extending an existing class, where the original is the base class and the new one is the derived class. Polymorphism: an object may respond differently when the same operation is invoked on it depending on the exact object to which the call is bound.

الترجمة: الكائن: كيان يقدر يحفظ حالة ويعرض عمليات لفحصها أو تعديلها. الرسالة: وسيلة تواصل الكائنات وتتكون من هوية الهدف واسم العملية المطلوبة وأي معلومات أخرى لازمة. التجريد: إزالة غير المهم وتضخيم الجوهري. الصنف: مجموعة كائنات تشارك بنية وسلوكًا مشتركين وتعمل كمخطط للكائنات. الوراثة: تعريف صنف جديد بتوسعة صنف قائم، حيث الأصل هو الصنف الأساسي والجديد هو الصنف المشتق. تعددية الشكل: الكائن قد يتصرف بشكل مختلف عند استدعاء نفس العملية عليه حسب الكائن الفعلي اللي يُربط به الاستدعاء.

5.17 What is the relationship between abstract data types and classes?

Answer: Every class is an abstract data type, but not every abstract data type is a class. An abstract data type hides the way data is stored internally and lists a set of allowed operations, so from the outside the data can only be accessed through well-defined methods. A class supports that, and additionally supports inheritance and the other object-orientation properties. Therefore a language that supports abstract data types but does not support inheritance is called an object-based language, not an object-oriented language, and Ada is an example of an object-based language. Classes are abstract data types because they can be instantiated into objects.

الترجمة: كل صنف هو نوع بيانات مجرد، لكن ليس كل نوع بيانات مجرد صنفًا. والنوع المجرد يخفي طريقة تخزين البيانات داخليًا ويسرد مجموعة عمليات مسموحة، فالوصول من الخارج يكون فقط عبر دوال مُعرّفة جيدًا. والصنف يدعم ذلك، وإضافةً يدعم الوراثة وبقية خصائص الكينونة. لذلك اللغة اللي تدعم أنواع البيانات المجردة لكن ما تدعم الوراثة تُسمى لغة قائمة على الكائنات مو كائنية التوجه، وAda مثال على ذلك. والأصناف أنواع بيانات مجردة لأنها تُنشأ منها كائنات.

5.18 Can we have inheritance without polymorphism? Explain.

Answer: Yes, inheritance is possible without polymorphism. Inheritance is a mechanism for reuse, where a derived class inherits the data and methods of its base class and may add new ones or redefine existing ones. Polymorphism, on the other hand, concerns how a single call is bound to different implementations. Inheritance without overriding produces no dynamic binding, so no polymorphism is present. The reverse relation, however, does not hold: a polymorphic operation leaves the implementation to subclasses, so polymorphism always presupposes inheritance, but inheritance does not presuppose polymorphism.

الترجمة: نعم، الوراثة ممكنة بلا تعددية شكل. فالوراثة آلية لإعادة الاستخدام حيث يورث الصنف المشتق بيانات ودوال الصنف الأساسي وقد يضيف جديدًا أو يعيد تعريف الموجود. أما تعددية الشكل فتتعلق بكيفية ربط استدعاء واحد بتنفيذات مختلفة. والوراثة بلا إعادة تعريف (overriding) ما تنتج ربطًا ديناميكيًا فلا توجد تعددية شكل. أما العكس فلا يصح: العملية متعددة الشكل تترك التنفيذ للأصناف المشتقة، فتعددية الشكل دائمًا تفترض الوراثة، لكن الوراثة ما تفترض تعددية الشكل.

5.19 Discuss the reasons for improvement using object-oriented design.

Answer: Object-oriented design improves software for several reasons. First, encapsulation or data hiding localises errors, because data can only be changed through a method, so any error can be traced to the code segment that changed the value. Second, an abstract-data-type based design displays high cohesion and low coupling, so object-oriented designs are highly modular. Third, objects communicate by message passing and do not directly change each other's data, so objects are weakly coupled, which enhances understandability because each object can be studied in isolation. Fourth, abstraction manages complexity, and fifth, dynamic binding and inheritance make code reuse and maintenance easier, since new derived objects can be added with minimal changes to a program. Object-oriented design focuses on the data manipulated by the program rather than the functions, so it is orthogonal to function-oriented design.

الترجمة: التصميم الكائني يحسّن البرمجيات لعدة أسباب: أولًا التغليف أو إخفاء البيانات يموضع الأخطاء لأن البيانات ما تتغير إلا عبر دالة فيسهل تتبع الخطأ للكود اللي غيّر القيمة، وثانيًا التصميم المبني على أنواع البيانات المجردة يُظهر ترابطًا عاليًا واقترانًا واطئًا لذا التصميم الكائني عالي التماسك، وثالثًا الكائنات تتواصل بتمرير الرسائل ولا تغيّر بيانات بعضها مباشرة فتصير ضعيفة الاقتران مما يزيد قابلية الفهم لأن كل كائن يُدرس منعزلًا، ورابعًا التجريد يدير التعقيد، وخامسًا الربط الديناميكي والوراثة يسهّلان إعادة الاستخدام والصيانة لأن أصنافًا مشتقة جديدة تُضاف بأقل تغيير بالبرنامج. والتصميم الكائني يركّز على البيانات اللي يتعامل معها البرنامج لا على الدوال، فهو متعامد مع التصميم الوظيفي.

5.20 Explain the design guidelines that can be used to produce "good quality" classes or reusable classes.

Answer: Several guidelines produce good quality or reusable classes. First, encapsulate the data so that the internal data of an object is private and is accessed only through its methods, which protects it and allows modification of the internal data or the method body without affecting other classes, leading to easier maintenance and bug correction. Second, keep high cohesion and low coupling so each class performs one clear responsibility. Third, use inheritance to place common features in a base class and reuse them. Fourth, prefer dynamic binding so the system can be extended by adding classes without changing existing code. Fifth, keep the interface simple and stable, because a class with few and simple interfaces is easy to reuse and to extract from one program and insert into another.

الترجمة: عدة إرشادات تنتج أصنافًا جيدة أو قابلة لإعادة الاستخدام: أولًا غلّف البيانات بحيث تكون داخلية خاصة ولا يُوصل إليها إلا عبر دوالها، وهذا يحميها ويسمح بتعديل البيانات الداخلية أو جسم الدالة بلا التأثير على الأصناف الأخرى فتسهل الصيانة وتصحيح الأخطاء، وثانيًا حافظ على ترابط عالٍ واقتران واطئ بحيث يؤدي كل صنف مسؤولية واحدة واضحة، وثالثًا استخدم الوراثة لوضع المشترك في الصنف الأساسي وإعادة استخدامه، ورابعًا فضّل الربط الديناميكي ليُوسَّع النظام بإضافة أصناف بلا تغيير الكود القائم، وخامسًا حافظ على واجهة بسيطة وثابتة لأن الصنف بواجهات قليلة وبسيطة يسهل إعادة استخدامه وسحبه من برنامج وإدخاله في آخر.

5.21 List the points of a simplified design process.

Answer: The simplified design process has these points: gather data on user requirements; analyse the requirements data; conceive of a high level design; refine and document the design; obtain answers to requirement questions; and validate the design against the requirements. The output of the process is a completed design. In parallel terms, software design is carried out through two stages, high-level design where the important components and their interactions are identified, and detailed design where the algorithms and data structures are identified.

الترجمة: عملية التصميم المبسطة هذه نقاطها: جمع بيانات متطلبات المستخدم، وتحليل بيانات المتطلبات، وتصور تصميم عالي المستوى، وتنقيح التصميم وتوثيقه، والحصول على أجوبة أسئلة المتطلبات، والتحقق من التصميم مقابل المتطلبات.

ومخرج العملية تصميم مكتمل. وبصيغة موازية، التصميم يمر بمرحلتين: التصميم عالي المستوى حيث تُحدَّد المكونات المهمة وتفاعلاتها، والتصميم التفصيلي حيث تُحدَّد الخوارزميات وبنى البيانات.

5.22 Discuss the differences between object oriented and function oriented design.

Answer: In function-oriented design the basic unit is the function or module, and functions appear as verbs in the problem description; the system state is centralised and shared among functions through global data. In object-oriented design the basic unit is the object, and objects appear as nouns; the state information is decentralised and distributed among objects, and the data internal to an object cannot be accessed directly by other objects and only through its methods. Function-oriented techniques group functions together if as a group they constitute a higher level function, while object-oriented techniques group functions together on the basis of the data they operate on. Object-oriented design focuses on the data manipulated by the program, and it is orthogonal to function-oriented design; however the two approaches are complementary rather than competing, because top-down function-oriented techniques are often applied to design the internal methods of a class.

الترجمة: في التصميم الوظيفي الوحدة الأساسية هي الدالة أو الموديول وتظهر الدوال كأفعال في وصف المشكلة، وحالة النظام مركزية ومشتركة بين الدوال عبر البيانات الشاملة. أما في التصميم الكائني فالوحدة الأساسية هي الكائن وتظهر الكائنات كأسماء، ومعلومات الحالة موزعة بين الكائنات، والبيانات داخل الكائن لا تُوصل إليها الكائنات الأخرى مباشرة بل عبر دوالها فقط. والتقنيات الوظيفية تجمع الدوال إذا كانت كمجموعة تشكّل دالة أعلى، أما الكائنية فتجمع الدوال حسب البيانات اللي تعمل عليها. والتصميم الكائني يركّز على البيانات اللي يتعامل معها البرنامج وهو متعامد مع التصميم الوظيفي، لكن النهجين متكاملان لا متنافسان، لأن تقنيات التصميم الوظيفي من الأعلى للأسفل كثيرًا ما تُطبَّق لتصميم الدوال الداخلية لصنف.

5.23 What documents should be produced on completion of the design phase?

Answer: The official document produced at the completion of the design phase is the Software Design Description (SDD), recommended by IEEE Standard 1016-1998. It is a representation of a software system used as a medium for communicating software design information. It shows how the software system will be structured to satisfy the requirements identified in the SRS, so it becomes the blueprint for the implementation activity. Its content includes scope and references, design entities and design entity attributes, and a set of design views: the decomposition description, the dependency description, the interface description and the detail description. The design entity attributes are identification, type, purpose, function, subordinates, dependencies, interface, resources, processing and data.

الترجمة: المستند الرسمي اللي يُنتَج عند إكمال مرحلة التصميم هو وصف التصميم البرمجي SDD، الموصى به في معيار IEEE 1016-1998. وهو تمثيل لنظام برمجي يُستخدم وسيطًا لتوصيل معلومات التصميم. ويوضح كيف ستُبنى البرمجيات لتحقق المتطلبات المحددة في مستند SRS، فيصير مخطط البناء لمرحلة التنفيذ. ومحتواه يشمل النطاق والمراجع، وكيانات التصميم وصفاتها، ومجموعة صيغ تصميم: وصف التفكيك، ووصف الاعتماد، ووصف الواجهة، ووصف التفصيل. وصفات كيان التصميم هي: التعريف والنوع والغرض والوظيفة والتابعون والاعتمادات والواجهة والموارد والمعالجة والبيانات.

5.24 Can a system ever be completely "decoupled"? That is, can the degree of coupling be reduced so much that there is no coupling between modules?

Answer: Theoretically yes, because an uncoupled state with no dependencies is recognised as a possibility. Practically no, because a useful system requires its modules to cooperate to deliver a single function, and a module with no coupling at all would be isolated and could not contribute to the behaviour of the system. The practical best case is low coupling, where two modules either do not interact at all or at best interact by passing no data or only a few primitive data items, and the minimum useful form is data coupling. In software design we strive for the lowest possible coupling, not for none.

الترجمة: نظريًا نعم، لأن الحالة غير المقترنة بلا اعتماد معترف بها كاحتمال. أما عمليًا فلا، لأن النظام النافع يتطلب تعاون وحداته لتقديم وظيفة واحدة، والوحدة بلا اقتران إطلاقًا تكون معزولة وما تساهم في سلوك النظام. والحد العملي الأفضل هو الاقتران الواطئ، حيث لا تتفاعل وحدتان إطلاقًا أو تتفاعلان بتمرير بيانات قليلة أو أولية فقط، وأقل شكل مفيد هو الاقتران بالبيانات. وفي التصميم نسعى لأقل اقتران ممكن لا إلى انعدامه.